

DEEP LEARNING PRACTICE

Quiz - 1

Week – 1

What is Natural Language Processing?

Enabling the computers to understand, interpret, and generate human language.

So that, given an input text, we can ask the computer to translate, sentiment classification, Named Entity Recognition (NER).

(Note: NER means, a NLP technique that identifies and classifies key elements – like people, organizations, locations in unstructured text).

Week - 2

What is tokenizer?

The tokenizer simply splits the input sequence into tokens. A simple approach is to use white space for splitting the text. Each token is associated with unique ID. And each ID is associated with a unique embedding vector. The model then takes these embeddings and predict the token ID's. During Inference, the predicted tokens are converted back to tokens (and then back to words).

The tokenizer essentially contains two components:

Encoder : converts input word to token ID.

Decoder : converts input token ID to word.

What is pre-tokenization?

The process of splitting the text into words using whitespace.

Challenges in building a Vocabulary:

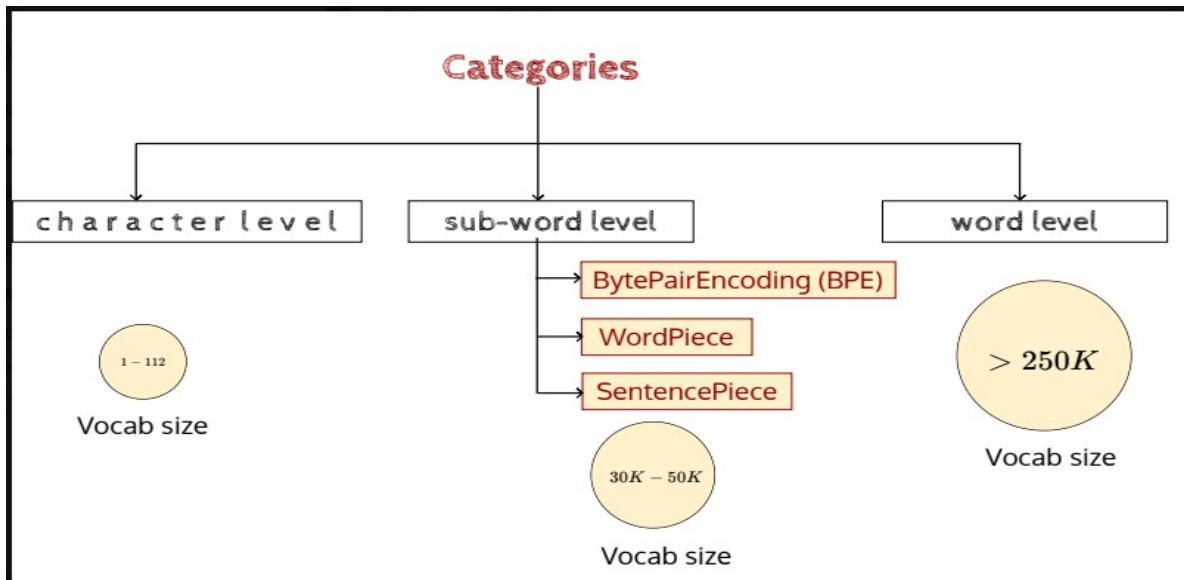
What should be the size of Vocabulary?

Out-of-vocabulary

Handling misspelled words in corpus

Open Vocabulary problem.

Tokenization Algorithms:



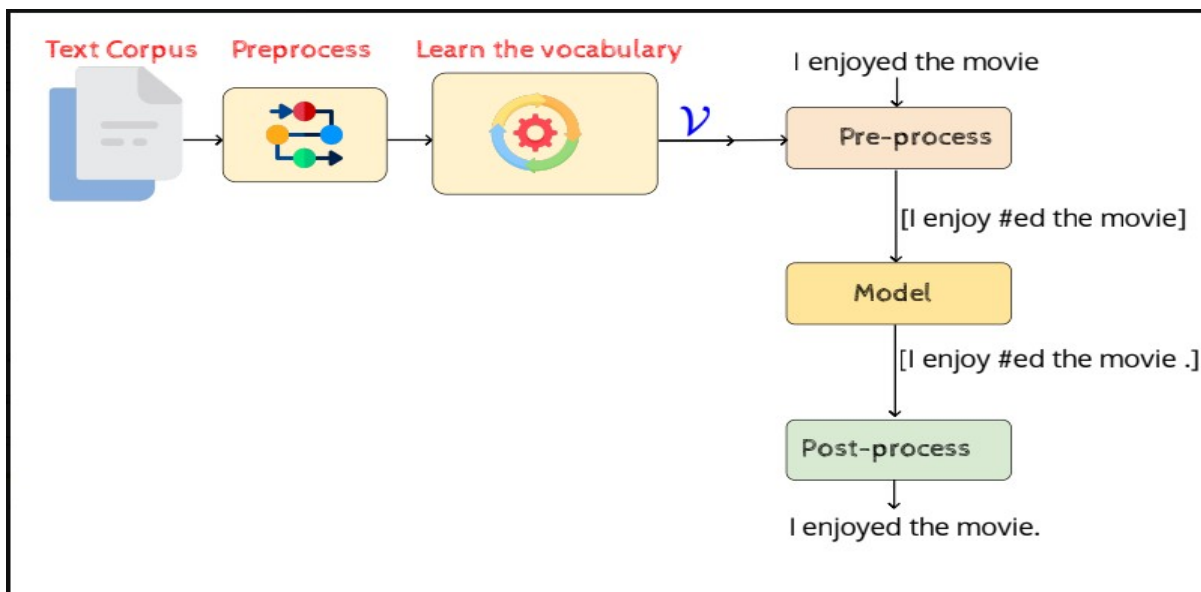
Byte Pair Encoding:

General Pre-Processing Steps:

First the text is normalized, which involved operations such as treating cases, removing accents, eliminating multiple whitespaces, handling HTML tags. Etc.

Splitting the whitespace was traditionally called tokenization. When it is used with a sub-word tokenization algorithm, it is called pre-tokenization.

Learning the vocab using these words is called training.



Algorithm:

Start with a dictionary that contains words and their counts.

Append a special symbol “</w>” at the end of each word in the dictionary.

Set the required number of merges (a hyperparameter).

Initialize the character-frequency table(base vocabulary).

Get the frequency count for a pair of characters.

Merge pair with maximum occurrence.

Final vocabulary contains initial vocabulary and all the merges (in order).

At test time, the input words is split into a sequence of characters, and the characters are merged into larger known symbols.

BPE for Non-segmented Languages:

Languages such as Japanese and Korean are non-segmented.

However, BPE requires space-separated words. So, In practice we can use language specific morphology based word segmenters such as Juman++ for Japanese. But in case of multilingual translation, having language agnostic tokenizer is desirable.

(Note: BPE is greedy and deterministic. We can use BPE-Dropout to make it stochastic).

WORD PIECE ALGORITHM:

what if there are more pairs that are occurring with the same frequency? Take the frequency of occurrence of individual tokens into account.

$$score = \frac{count(\alpha, \beta)}{count(\alpha) \times count(\beta)}$$

The WordPiece algorithm uses this score to merge the pairs.

SENTENCE PIECE ALGORITHM:

1. Construct a reasonably large size vocabulary using BPE or Extended Suffix Array Algorithm.

2. E-step:

Estimate the probability for every token in the given vocabulary using frequency counts in the training corpus.

3. M-step:

Use Viterbi algorithm to segment the corpus and return optimal segments that maximizes the (log) likelihood.

4. Compute the likelihood for each new sub-word from optimal segments.

5. Shrink the vocabulary size by removing top x% of sub-words that have the smallest likelihood.

6. Repeat step 2 to 5 until desired vocabulary size is reached.

The best thing about the algorithm is, its Language Agnostic. So we can use this algorithm for languages that do not use any word delimiters in a sentence.

WEEK – 3

Let V be a vocabulary of language (i.e., collection of all unique words in the language).

For example, $V = \{\text{an, ate, apple, I}\}$, some possible sentences (not necessarily grammatically correct) are,

a. An apple ate I

b. I ate an apple

c. I ate apple

d. an apple

e.

Intuitively, some of these sentences are more probable than others. We mean that, given a very very large corpus, we expect some of these sentences to appear more frequently than others.

We are not looking for a function which takes a sequence as input and assigns a probability to each sequence.

$$f : (X_1, X_2, \dots, X_n) \rightarrow [0, 1]$$

such a function is called **Language Model**.

Roughly speaking, this task of predicting the next token in a sequence is called language modelling.

Auto regressive models does this. And we say it as Causal Language Modelling (CLM). Given a sequence of text, it (model) learns to predict the next token in sequence.

WEEK – 4

Parameter Efficient Fine Tuning (PEFT) :

LoRa, QLoRa and AdaLoRa are most commonly used (optionally, in combination with quantization).

Emerging Abilities:

Fine-tuning large models is costly, as it still requires thousands of samples to perform well in a downstream task.

Some task may not have enough labeled samples. Moreover, this is not how humans adapt to different tasks once they understand the language.

We can give them a book and “**prompt**” them to summarize it or find an answer for a specific question. That is, we do not need “**supervised fine tuning**” at all.

In-context Learning:

The technique in which the model learns a new task from samples within the context window, this approach is called “**In-context**” learning.

This remarkable ability enables the adaption of the model to downstream tasks without the need for gradient based fine tuning. Adaption happens on the fly in inference mode.

Prompting:

There are many ways of prompting the model. For example,

1. Zero-shot.
2. Few-shot (in-context).
3. Chain of Thought.
4. Prompt Chaining.

Instruction Tuning:

zero-shot learning performance is often poor, despite the model size (say, GPT3 175B), especially in following the user instructions.

How do we improve the zero-shot learning performance?

Fine tune the model on the instructions.

Preference Tuning:

Uses the collections to fine-tune (Supervised Fine Tuning, SFT) the model using Reinforcement Learning from Human Feedback (RLHF).

What is RL?

Its as branch of machine learning, that focuses on how agents can learn to make decisions through trial and error to maximize cumulative rewards.